

Inlämning 2

Losningen och kritisk reflektion

TeacherAid — AI-stöd för lärare på Yrkesakademin

Student: Kim Safsten | Kurs: SYS25D | Deadline: 21 juni 2026

Denna rapport redovisar säkerhetsanalysen och den kritiska reflektionen för TeacherAid-projektet. Koden, automationen och README är tillgängliga via det inlämnade GitHub-repot.

Losningen är en RAG-baserad webbapplikation som hjälper lärare att generera feedbackutkast och besvara kursfrågor med hjälp av lokal AI (Ollama). All behandling sker på skolans egna serverar — ingen studentdata lämnar organisationens kontroll.

Del 1: Sakerhetsanalys

Mappad mot OWASP LLM Top 10 (2025)

OWASP LLM Top 10 är en referenslista över de tio vanligaste sakerhetsriskerna i system som använder stora språkmodeller. Nedanstående analys går igenom varje risk, bedomer dess relevans för TeacherAid och beskriver hur den hanteras eller varför den är begränsad i detta system.

Riskmatris

Varje risk bedoms på en femgradig skala för både sannolikhet och konsekvens. Score = Sannolikhet x Konsekvens. Niva: HOG (score ≥ 12), MEDEL (5-11), LAG (≤ 4).

Risk	Sannolikhet (1-5)	Konsekvens (1-5)	Score	Niva
LLM09 - Misinformation	4	5	20	HOG
LLM01 - Prompt Injection	3	4	12	HOG
LLM05 - Improper Output Handling	3	3	9	MEDEL
LLM02 - Sensitive Info Disclosure	2	4	8	MEDEL
LLM08 - Vector/Embedding Weakness	2	3	6	MEDEL
LLM07 - System Prompt Leakage	2	2	4	LAG
LLM06 - Excessive Agency	1	3	3	LAG
LLM04 - Data & Model Poisoning	1	3	3	LAG
LLM03 - Supply Chain	1	2	2	LAG
LLM10 - Unbounded Consumption	1	1	1	LAG

LLM01 — Prompt Injection [Score: 12 | Niva: HOG]

Beskrivning: En student kan formulera sin inlämning på ett sätt som förstör manipulera AI-modellen att ignorera instruktionerna från läraren — till exempel att begära ett högre betyg än som är motiverat, eller att extrahera information om andra studenters inlämningar.

Var uppstår risken i TeacherAid: Risken uppstår i n8n-workfloden där studentens inlämningstext skickas rakt in i prompten till llama3. Innehållet sanitiserats inte fullt ut innan det kopplas samman med instruktionerna.

Åtgärd: (1) Implementera SK-02: rensa output från kontrollkaraktär och null-bytes, begränsat till 8 000 tecken. (2) Strukturera prompten så att studenttexten är tydligt avgränsad med en fast avgränsare, t.ex. `###STUDENT_TEXT_START###...###STUDENT_TEXT_END###`, så att instruktionerna aldrig kan överskridas av studentinnehåll. (3) Logga och flagga inlämningar där output avviker markant från förväntad feedback-format.

Avvagning: Strangare sanitisering kan ta bort legitima specialtecken i programmeringsinlämningar. Avvagning: rensa kontrolltecken men bevara ASCII-grafik och kodblock som är relevanta för kursen.

LLM09 — Misinformation [Score: 20 | Niva: HOG]

Beskrivning: AI-modellen kan generera sakligt felaktig feedback som läraren godkänner utan att granska den noggrant. För en student kan det innebära att de får en IG på ett korrekt svar eller att de tror att en felaktig lösning är godkänd.

Var uppstår risken i TeacherAid: llama3 (7B-modell) är begränsad i facktermkunskap. Modellen är inte finjusterad på pedagogisk bedomning och kan hallucera kriterier som inte finns i uppgiftsbeskrivningen. Risken är störst när kursmaterialet i RAG-indexet är glest eller föråldrat.

Åtgärd: (1) All feedback presenteras explicit som 'AI-utkast' i UI:t — läraren måste aktivt klicka 'Godkänn och spara', aldrig en automatisk publicering. (2) AutomationLog loggar varje AI-anrop så att man kan spara vilken version av materialet som användes vid bedomningen. (3) Utöka kursmaterialet i RAG-indexet så att chunksarna täcker alla bedomningskriterier — minskar hallucineringsrisken.

Avvagning: Att kräva läraens godkännande på varje feedback eliminerar tidsvinsten om granskningssteget tar lika lång tid som att skriva feedback från noll. Avvagning: AI:n sparar tid på att generera ett strukturerat utkast — läraren fokuserar på att validera, inte på att formulera från grunden.

LLM05 — Improper Output Handling [Score: 9 | Niva: MEDEL]

Beskrivning: AI-genererad output som innehåller skadliga teckenföljder, null-bytes eller extremt långa strängar kan orsaka problem när texten sparas i databasen, visas i UI:t eller skickas vidare i automationsfloden.

Var uppstår risken i TeacherAid: `OllamaLLMService.GenerateAsync()` returnerar modellens rådata svar direkt till databasen och frontendens textarea. Ingen sanitisering implementerades i produktion (SK-02 återställdes av linter).

Åtgärd: Återimplementera SK-02 i `OllamaLLMService.GenerateAsync()`: ta bort kontrolltecken (`\u0000-\u001f` utom `\n`, `\r`, `\t`), begränsa sträng till 8 000 tecken, och validera att returvärdet är giltigt UTF-8. Detta bör behandlas som P0 inför produktionssättning.

Avvagning: Negligibel — sanitisering påverkar inte kvaliteten på legitim feedback. Kostnaden är en extra strängoperation per anrop.

LLM02 — Sensitive Information Disclosure [Score: 8 | Niva: MEDEL]

Beskrivning: Studentinlämningar innehåller personuppgifter. Utan pseudonymisering skulle dessa skickas till AI-modellen i klartext och potentiellt kunna rekonstrueras ur embedding-vektorer i databasen.

Var uppstår risken i TeacherAid: `FolderSyncService.SyncInlämningar()` parser studentnamnet ur filnamnet och använder det för att pseudonymisera texten. Mappningstabellen (studentnamn $\leftrightarrow$ pseudonym) lagras implicit i `Submission.StudentName` och är i sig persondata.

Atgard: (1) Pseudonymisering implementerad: TextPseudonymizer ersatter namn med [Student] i allt material som skickas till AI. (2) Lokal Ollama gor att inga personuppgifter lamnar organisationens infrastruktur. (3) Rollbaserad atkomstkontroll (JWT) sakerstaller att enbart behoriga larare ser studentdata. (4) Som nasta steg: kryptera Submission.Content i vilan (encryption at rest).

Avvagning: Pseudonymisering baserad pa filnamn missar fall dar studenten namnger sig sjalv i texten med ett namn som inte matchar filnamnet. En mer robust losning skulle anvanda NER (named-entity recognition), men det kraver en separata NLP-modell och okar komplexiteten markant.

LLM08 — Vector and Embedding Weaknesses [Score: 6 | Niva: MEDEL]

Beskrivning: Embedding-vektorer lagras i pgvector utan kryptering. En databas-dump skulle exponera vektorer som kan anvandas for att rekonstruera delar av det indexerade materialet.

Var uppstar risken i TeacherAid: DocumentChunk.Embedding lagras som vector(768) i PostgreSQL. Chunking-algoritmen splittar enbart pa radbrytningar, vilket ger inkonsekvent chunk-storlek och kan ge svar med lag relevans.

Atgard: (1) Begransat till lokal infrastruktur — pgvector-databasen ar inte exponerad mot internet. (2) Forbatta ChunkText (nu TextChunker.Chunk()) att aven splitta pa meningsslut for jamnare chunks. (3) Langsiktig atgard: aktivera pgcrypto for kryptering av kansliga kolumner.

Avvagning: Semantisk chunking pa meningsslut kraver sprakanalys och ger marginell vinst i ett MVP-skede. Prioriteras efter att grundfunktionaliteten ar stabil.

LLM07 — System Prompt Leakage [Score: 4 | Niva: LAG]

Beskrivning: Kursmaterialet i RAG-kontexten kan potentiellt extraheras om en student staller fragor som ar utformade att fa AI:n att citera kontexten ordagrant.

Var uppstar risken i TeacherAid: QaController.AskQuestion() inkluderar upp till tre RAG-chunks i prompten utan redaktion.

Atgard: Risken ar begransad eftersom QA-endpointen ar skyddad av [Authorize] och enbart tillganglig for inloggade larare i MVP-versionen. Om funktionen oppnas for studenter: lagg till instruktion i system-prompten att aldrig citera kontexten ordagrant.

Avvagning: Ingen avvagning kravs i nuvarande MVP-scope.

LLM06 — Excessive Agency [Score: 3 | Niva: LAG]

Beskrivning: AI-modellen tar autonoma atgarder utan laraens godkannande, till exempel skickar feedback direkt till studenter eller andra betyg i systemet.

Var uppstar risken i TeacherAid: Inte applicerbart i nuvarande design.

Atgard: Systemet ar medvetet designat utan agentic agency: AI genererar enbart utkast, lararen maste explicit godkanna via PUT /api/submissions/{id}/feedback innan nagonting sparas som slutgiltigt aterkoppling. Inga autonoma utskick implementerade.

Avvagning: Design-valet eliminerar risken helt pa bekostnad av automatiseringsgrad.

LLM04 — Data and Model Poisoning [Score: 3 | Niva: LAG]

Beskrivning: En aktör med skrivatkomst till kursmaterialsmappen kan ladda upp manipulerade dokument som förvränger AI:ns svar på kursfrågor.

Var uppstar risken i TeacherAid: kursmaterial/-mappen synkroniseras automatiskt vid POST /api/sync/kursmaterial.

Atgard: JWT-autentisering begransar synk-endpointen till inloggad lärare. Filformat valideras (enbart .pdf, .docx, .txt). Som nästa steg: logga alla synkoperationer och implementera filhash-verifiering.

Avvagning: En organisation med en enda läraranvändare har minimalt hot-surface.

LLM03 — Supply Chain Vulnerabilities [Score: 2 | Niva: LAG]

Beskrivning: Komprometterade beroenden (NuGet-paket, Docker-images) kan introducera skadlig kod i systemet.

Var uppstar risken i TeacherAid: Alla beroenden: PdfPig, DocumentFormat.OpenXml, Pgvector, n8n, Ollama.

Atgard: Lokal Ollama i Docker eliminerar beroendet av externa AI-API:er. Langsiktig atgard: aktivera GitHub Dependabot för automatiska säkerhetsupdateringar av NuGet-paket.

Avvagning: Standard DevSecOps-praxis, ingen specifik avvagning.

LLM10 — Unbounded Consumption [Score: 1 | Niva: LAG]

Beskrivning: Obegransad användning av AI-resurser leder till prestanda-degradering eller okontrollerade kostnader.

Var uppstar risken i TeacherAid: Ollama kors lokalt — inga token-kostnader per anrop.

Atgard: Lokal inferens eliminerar den ekonomiska risken. Prestandabegransning hanteras av serverns GPU/CPU-kapacitet. Rate limiting kan laggas till på API-niwa om behovet uppstar.

Avvagning: Ingen avvagning krävs i nuvarande skede.

Del 2: Kritisk reflektion

Jag hade 4 veckor på mig att bygga TeacherAid från beslutsunderlag till fungerande produkt. Nedan reflekterar jag över vad som fungerade, vad som inte fungerade, och vad jag skulle göra annorlunda — både i detta projekt och i AI-assisterad systemutveckling generellt.

Vad fungerade

ILLMService-abstraktionen

Att extrahera ett ILLMService-interface tidigt var det enskilt bästa arkitekturbeslutet i projektet. Det innebär att jag kunde byta ut Ollama mot en annan modell utan att röra några controllers, och att enhetstesterna kunde mocka AI-lagret utan att kreira en kord Ollama-instans. Claude foreslog det här monster, och det visade sig vara korrekt.

Lokal AI eliminerade GDPR-komplexiteten

Beslutet att kora Ollama lokalt innebär att ingen studentdata lämnar skolans infrastruktur. I Inlämning 1 beskrev jag behovet av ett personuppgiftsbitrades-avtal (DPA) om data skickas till externa AI-tjänster. Genom att välja lokal inferens tog jag bort det kravet från MVP helt. Det är ett designval som AI:n inte kan göra automatiskt — det krävde att jag forstod juridiken och kopplade den till arkitekturvalet.

n8n för automationen

n8n fungerade väl för att prototypa feedback-automationen snabbt. Webhook-triggern, Ollama-anropet och SQL-insatsen konfigurerades utan att skriva kod. När SQL-injektionsfelet duppade (apostrofer från AI-output bröt raw SQL) lokaliserade jag felet till n8n-noden och fixade det med parameteriserade fragor — ett misstag som AI:n hade missat men jag fangade i testning.

Enhetstesterna tvingade fram bra design

Att skriva 18 enhetstester för TextPseudonymizer, SubmissionFileNameParser och TextChunker krävde att dessa extraherades från FolderSyncService och RagService till egna statiska klasser. Resultatet blev kod som är lättare att testa, lättare att läsa, och lättare att byta ut. AI genererade testerna snabbt, men det var jag som måste identifiera vilket beteende som faktiskt var värt att testa (kortnamnsundantaget, överstorleksparagrafen, ParseCourseId-fallets nollfallet).

Vad fungerade inte

SK-02 (output-sanitisering) återställdes av linter

Den viktigaste säkerhetsfunktionen — att rensa AI-output från kontrolltecken — återställdes av en automatisk linter-korrigerings och hamnade aldrig i produktion. Det avslöjade en brist i min arbetsprocess: jag granskade inte vad lintern ändrade. En kompilatörvarning är inte samma sak som en granskad pull request.

Chunking-algoritmen delar inte på meningar

TextChunker.Chunk() splittar enbart på radbrytningar. En lång mening utan \n hamnar i ett enda chunk som kan överskrida 500 tecken. Det innebär att RAG-sökningen ibland returnerade chunks som var för stora för att ge precis stöd till AI:ns svar. Jag kände till begränsningen (den är

dokumenterad i README) men laste den inte under MVP-fasen.

Filnamnskonventionen är braktig

Uppgiftsbeskrivning_inl2_SYS25D.txt tolkades av SubmissionFileNameParser som kurs-ID 'inl2' istället för 'SYS25D'. Felet berodde på att parsern använder det andra understreck-segmentet (parts[1]), och jag namngav filen fel. Det har inga konsekvenser i kod — det är ett organisatoriskt problem. En robust lösning hade validerat att kurs-ID matchar ett kant monster (t.ex. regex `[A-Z]{3}\d{2}[A-Z]`) och flaggat ovanstående format.

Vad jag skulle gora annorlunda

Borja med sakerheten, inte med funktionaliteten

Jag implementerade ILLMService och RagService innan jag tankte igenom sakerhetskraven. SK-02 las till i efterhand och återställdes. Om jag hade börjat med en OWASP LLM-analys och implementerat sanitiseringen som en första service (inte en efterkonstruktion) hade den aldrig återställts — den hade varit en del av kontraktet från start.

Anvant migrations från dag ett

Jag började med `db.Database.EnsureCreated()` och bytte later till `db.Database.Migrate()`. Kombinationen av bada orsakade ett konfliktfel som behövde felsöku. I ett riktigt projekt hade jag anvant Code First migrations från dag ett och aldrig blandade dem.

Skrivit integrationstester, inte bara enhetstester

De 18 enhetstesterna testar isolerad affärlogik, men ingen test verifierar att SyncController faktiskt synkar filer till databasen, att n8n-webhooken tar emot data korrekt, eller att RAG-sökningen returnerar relevanta chunks. Integrationstester med en test-databas och mockat Ollama hade gett högre säkerhet vid refaktorering.

När är AI rätt verktyg i systemutveckling?

Slutsatsen från det här projektet häller utöver TeacherAid:

AI är ett effektivt verktyg när:

Uppgiften är väl-definierad med tydliga framgångsskriterier — generera en klass, skriva ett test, översätta ett felmeddelande. AI är snabb och korrekt när det finns ett facit att jämföra mot. Exempel: ILLMService-interfacet, enhetstesterna, XML-dokumentationen. Detsamma gäller boilerplate-intensiva uppgifter: controller-skelett, DTO-klasser, EF Core-konfiguration — AI sparar tid på allt som följer ett förutsagbart monster.

AI är ett riskfyllt verktyg när:

Beslutet kräver kontext som AI:n saknar: juridisk bedömning (GDPR och DPA-kravet), affärslogik (att 'inl2' är fel kurs-ID), eller säkerhetskrav (att SK-02 inte fick återställas). I alla tre fallen krävdes mänsklig bedömning som inte kan delegeras. AI är också opålitlig när det gäller att bevara invarianter över tid — den kan introducera en kod-ändring som bryter en säkerhetsegenskap som introducerades tre commits tidigare utan att känna till den.

Slutsats:

AI i systemutveckling fungerar bäst som en snabb, tolerant pair-programmerare med bredare kunskap än de flesta men utan förmåga att förstå principer eller fatta kontextkänsliga beslut. Ansvar, arkitektur och säkerhet måste gå av en människa. Den som delegerar dessa till AI får ett system som ser korrekt ut men inte är det.

Länk till GitHub-repo och README finns i inlämningskommentaren.